JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Temporal

All Superinterfaces:

`TemporalAccessor`

All Known Subinterfaces:

`ChronoLocalDate`, `ChronoLocalDateTime<D>`, `ChronoZonedDateTime<D>`

All Known Implementing Classes:

`HijrahDate`, `Instant`, `JapaneseDate`, `LocalDate`, `LocalDateTime`, `LocalTime`, `MinguoDate`, `OffsetDateTime`, `OffsetTime`, `ThaiBuddhistDate`, `Year`, `YearMonth`, `ZonedDateTime`

```
public interface Temporal
extends TemporalAccessor
```

Framework-level interface defining read-write access to a temporal object, such as a date, time, offset or some combination of these.

This is the base interface type for date, time and offset objects that are complete enough to be manipulated using plus and minus. It is implemented by those classes that can provide and manipulate information as fields or queries. See `TemporalAccessor` for the read-only version of this interface.

Most date and time information can be represented as a number. These are modeled using `TemporalField` with the number held using a `long` to handle large values. Year, month and day-of-month are simple examples of fields, but they also include instant and offsets. See `ChronoField` for the standard set of fields.

Two pieces of date/time information cannot be represented by numbers, the chronology and the time-zone. These can be accessed via `queries` using the static methods defined on `TemporalQuery`.

This interface is a framework-level interface that should not be widely used in application code. Instead, applications should create and pass around instances of concrete types, such as `LocalDate`. There are many reasons for this, part of which is that implementations of this interface may be in calendar systems other than ISO. See `ChronoLocalDate` for a fuller discussion of the issues.

When to implement

A class should implement this interface if it meets three criteria:

- it provides access to date/time/offset information, as per `TemporalAccessor`

- the set of fields are contiguous from the largest to the smallest
- the set of fields are complete, such that no other field is needed to define the valid range of values for the fields that are represented

Four examples make this clear:

- `LocalDate` implements this interface as it represents a set of fields that are contiguous from days to forever and require no external information to determine the validity of each date. It is therefore able to implement plus/minus correctly.
- `LocalTime` implements this interface as it represents a set of fields that are contiguous from nanos to within days and require no external information to determine validity. It is able to implement plus/minus correctly, by wrapping around the day.
- `MonthDay`, the combination of month-of-year and day-of-month, does not implement this interface. While the combination is contiguous, from days to months within years, the combination does not have sufficient information to define the valid range of values for day-of-month. As such, it is unable to implement plus/minus correctly.
- The combination day-of-week and day-of-month ("Friday the 13th") should not implement this interface. It does not represent a contiguous set of fields, as days to weeks overlaps days to months.

Implementation Requirements:

This interface places no restrictions on the mutability of implementations, however
immutability is strongly recommended. All implementations must be `Comparable`.

Since:

1.8

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`boolean`	**`isSupported`**`(TemporalUnit unit)`	Checks if the specified unit is supported.
`default Temporal`	**`minus`**`(long amountToSubtract, TemporalUnit unit)`	Returns an object of the same type as this object with the specified period subtracted.
`default Temporal`		

	minus (TemporalAmount amount)	Returns an object of the same type as this object with an amount subtracted.
Temporal	**plus**(long amountToAdd, TemporalUnit unit)	Returns an object of the same type as this object with the specified period added.
default Temporal	**plus** (TemporalAmount amount)	Returns an object of the same type as this object with an amount added.
long	**until** (Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another temporal in terms of the specified unit.
default Temporal	**with** (TemporalAdjuster adjust	Returns an adjusted object of the same type as this object with the adjustment made.
Temporal	**with** (TemporalField field, long newValue)	Returns an object of the same type as this object with the specified field altered.

Methods declared in interface TemporalAccessor

get, getLong, isSupported, query, range

Method Details

isSupported

boolean isSupported(TemporalUnit unit)

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date-time. If false, then calling the plus(long, TemporalUnit) and minus methods will throw an exception.

Implementation Requirements:

Implementations must check and handle all units defined in `ChronoUnit`. If the unit is supported, then true must be returned, otherwise false must be returned.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

Parameters:

`unit` - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

with

```
default Temporal with(TemporalAdjuster adjuster)
```

Returns an adjusted object of the same type as this object with the adjustment made.

This adjusts this date-time according to the rules of the specified adjuster. A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

Some example code indicating how and why this method is used:

```
date = date.with(Month.JULY);        // most key classes implement TemporalAdju
date = date.with(lastDayOfMonth());  // static import from Adjusters
date = date.with(next(WEDNESDAY));   // static import from Adjusters and DayOfW
```

Implementation Requirements:

Implementations must not alter either this object or the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

The default implementation must behave equivalent to this code:

```
return adjuster.adjustInto(this);
```

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

with

```
Temporal with(TemporalField field,
              long newValue)
```

Returns an object of the same type as this object with the specified field altered.

This returns a new object based on this one with the value for the specified field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Implementation Requirements:

Implementations must check and handle all fields defined in `ChronoField`. If the field is supported, then the adjustment must be performed. If unsupported, then an `UnsupportedTemporalTypeException` must be thrown.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the first argument.

Implementations must not alter this object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

an object of the same type with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

plus

```
default Temporal plus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount added.

This adjusts this temporal, adding according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
date = date.plus(period);               // add a Period instance
date = date.plus(duration);             // add a Duration instance
date = date.plus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Implementation Requirements:

Implementations must not alter either this object or the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

The default implementation must behave equivalent to this code:

```
return amount.addTo(this);
```

Parameters:

`amount` - the amount to add, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if the addition cannot be made

ArithmeticException - if numeric overflow occurs

plus

```
Temporal plus(long amountToAdd,
              TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period added.

This method returns a new object based on this one with the specified period added. For example, on a `LocalDate`, this could be used to add a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then adding one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Implementation Requirements:

Implementations must check and handle all units defined in `ChronoUnit`. If the unit is supported, then the addition must be performed. If unsupported, then an `UnsupportedTemporalTypeException` must be thrown.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the first argument.

Implementations must not alter this object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

Parameters:

`amountToAdd` - the amount of the specified unit to add, may be negative

`unit` - the unit of the amount to add, not null

Returns:

an object of the same type with the specified period added, not null

Throws:

DateTimeException - if the unit cannot be added

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

minus

```
default Temporal minus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount subtracted.

This adjusts this temporal, subtracting according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
date = date.minus(period);               // subtract a Period instance
date = date.minus(duration);             // subtract a Duration instance
date = date.minus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Implementation Requirements:

Implementations must not alter either this object or the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

The default implementation must behave equivalent to this code:

```
return amount.subtractFrom(this);
```

Parameters:

`amount` - the amount to subtract, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
default Temporal minus(long amountToSubtract,
                       TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period subtracted.

This method returns a new object based on this one with the specified period subtracted. For example, on a `LocalDate`, this could be used to subtract a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st March, then subtracting one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Implementation Requirements:

Implementations must behave in a manor equivalent to the default method behavior.

Implementations must not alter this object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

The default implementation must behave equivalent to this code:

```
return (amountToSubtract == Long.MIN_VALUE ?
    plus(Long.MAX_VALUE, unit).plus(1, unit) : plus(-amountToSubtract, unit));
```

Parameters:

`amountToSubtract` - the amount of the specified unit to subtract, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

an object of the same type with the specified period subtracted, not null

Throws:

`DateTimeException` - if the unit cannot be subtracted

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

until

```
long until(Temporal endExclusive,
           TemporalUnit unit)
```

Calculates the amount of time until another temporal in terms of the specified unit.

This calculates the amount of time between two temporal objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified temporal. The end point is converted to be of the same type as the start point if different. The result will be negative if the end is before the start. For example, the amount in hours between two temporal objects can be calculated using `startTime.until(endTime, HOURS)`.

The calculation returns a whole number, representing the number of complete units between the two temporals. For example, the amount in hours between the times 11:30 and 13:29 will only be one hour as it is one minute short of two hours.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, unit);
amount = unit.between(start, end);
```

The choice should be made based on which makes the code more readable.

For example, this method allows the number of days between two dates to be calculated:

```
long daysBetween = start.until(end, DAYS);
// or alternatively
long daysBetween = DAYS.between(start, end);
```

Implementation Requirements:

Implementations must begin by checking to ensure that the input temporal object is of the same observable type as the implementation. They must then perform the calculation for all instances of `ChronoUnit`. An `UnsupportedTemporalTypeException` must be thrown for `ChronoUnit` instances that are unsupported.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

In summary, implementations must behave in a manner equivalent to this pseudo-code:

```
// convert the end temporal to the same type as this class
if (unit instanceof ChronoUnit) {
```

```
    // if unit is supported, then calculate and return result
    // else throw UnsupportedTemporalTypeException for unsupported units
  }
  return unit.between(this, convertedEndTemporal);
```

Note that the unit's between method must only be invoked if the two temporal objects have exactly the same type evaluated by `getClass()`.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

Parameters:

`endExclusive` - the end temporal, exclusive, converted to be of the same type as this object, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this temporal object and the specified one in terms of the unit; positive if the specified object is later than this one, negative if it is earlier than this one

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to the same type as this temporal

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.